

A Brief Introduction to Programming with R - Exercises

MEcon & MiQE/F Introductory Week

Ana Armendariz & Federica Mascolo

2026-09-04

Part I: Background / Tools

Exercise A: Set up a working environment

1. Open **RStudio** and navigate to your desired working folder
 2. Create a new folder called **r_course**
 3. Set it as your **working directory**
 4. Create subfolders: **data** and **code**
- (*) Create a new **R Project** in the `r_course` folder
- (*) Choose your preferred RStudio **appearance**: Tools > Global Options > Appearance

Exercise B: R scripts

1. In the R console, type:

```
print("Hello world")
```

2. Create a new **R script**: File > New File > R Script (or Ctrl+Shift+N)
3. Type the same code in the script and **run it** using Ctrl+Enter

Part II: First Steps and Basic Concepts

Exercise A: Write a sum function

Write a function that takes a **numeric vector** as input and returns the **sum** of the vector's elements.

```
my_sum <- function(x) {  
  # Your code here  
}
```

Test by comparing with built-in `sum()` function.

Exercise B: Robustness and warnings

Test your `my_sum()` function with:

```
numbers2 <- c("1", "2", "3")
```

Add **error checking** to make the function robust.

Exercise C: Standard deviation function

Implement a function to compute the **standard deviation**:

$$\text{SD} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Exercise D: Standard error function

Building on Exercise C, implement:

$$SE_{\bar{x}} = \frac{SD}{\sqrt{N}}$$

Exercise E: One-sample t-test

Implement a **one-sample t-test** function:

$$t = \frac{\bar{x} - \mu_0}{SE_{\bar{x}}}$$

Exercise F: Fibonacci sequence

Generate the first 30 **Fibonacci numbers** where:

- $F_0 = 0, F_1 = 1$
- $F_n = F_{n-1} + F_{n-2}$ for $n > 1$

Exercise G: Multiples of 3 and 5 (*)

Write a function that computes the sum of all **multiples of 3 or 5** up to a number N .

Hint: Multiples of both 3 and 5 should only be added once!

Exercise H: Prime numbers (*)

Write a function that computes the sum of all **prime numbers** up to a number N .

Part III: Working with Data

Data wrangling with dplyr

Exercise I.1

- Load the dataset with `data(swiss)`
- Use `mutate()` to create two new variables:
 - `UrbanizationRate`: the inverse of `Agriculture` ($100 - \text{Agriculture}$)
 - `FertilityCategory`: a factor variable with levels:
 - 0–60: “Low”
 - 60–80: “Medium”
 - 80–100: “High”
- Hint: use `case_when()` for categorization

Exercise 1.2

- Filter observations where `Fertility > 70`
- Arrange results by `Education` in descending order

Exercise I.3

- Group data by `FertilityCategory`
- Calculate average Education for each category using `group_by()` and `summarize()`

Exercise I.4

- Find the top three observations with the highest `Fertility` values
- Display only `Fertility` and `Education` columns

Exercise 1.5

- Create `EducationToFertilityRatio = Education / Fertility`
- Show all observations where the ratio is between 0.1 and 0.2

Exercise I.6

- Group by `FertilityCategory`
- Calculate:
 - Median `Fertility`
 - Standard deviation of `Education`
- Store the results in a new data frame

Exercise I.7

- Use `n_distinct()` to determine the number of distinct values in:
 - `Education`
 - `FertilityCategory`

Data visualization with ggplot2

Exercise J.1

- Load the dataset `swiss.csv` or call `data(swiss)`

Exercise J.2

- Print the last three observations of the dataset
- Create summary statistics for all variables

Exercise J.3

- Create new variables:
 - `UrbanizationRate = 100 - Agriculture`
 - `FertilityCategory`:
 - 0–60: “Low”
 - 60–80: “Medium”
 - 80–100: “High”
- Hint: use `cut()`

Exercise J.4

- Create a histogram of Fertility
 - Fill bars in dark blue
 - Add a minimal theme

Exercise J.5

- Create a boxplot of the variable Education

Exercise J.6

- Create a scatter plot of Education vs. Fertility
- Add a regression line (linear model)

Exercise J.7

- Create scatter plots of Agriculture vs. Examination
- Show them separately for each FertilityCategory using `facet_wrap()`